ENGF0001 Anomaly Detection in MQTT Streams

Stephen Hailes

October 2026

Why anomaly detection?

Modern control and monitoring systems must remain safe and efficient even when sensors or actuators start to fail. In chemical and biological processes such as the ENGF0001 bioreactor, a fault in a sensor or an actuator can cause the temperature, pH or stirring rate to drift away from the desired values. Control loops will react to this, but they can only react to what their sensors tell them. If the temperature sensor reads lower than the true temperature, the controller will heat the liquid until the faulty reading matches the setpoint, by which time the real temperature is higher than it should be. If the fault then clears, the reading jumps to the true, overheated, value and the controller has to wait for the reactor to cool. You can see this in one of my slides.

Traditional alarms use fixed thresholds and may miss subtle or evolving faults. **Anomaly detection** aims to recognise when system behaviour no longer matches the "normal" baseline, using only the measured signals. In this exercise you will implement and test such a detector using live process data from a simulator that reproduces the physical dynamics, sensors, actuators and common failure modes of the real system.

If you run into any problems, please post on Moodle.

Exercise overview

You are going to spend this term building your own bioreactor, so it will not be ready in time to provide data for this exercise. Instead, we provide streams of simulated data. This gives everyone the same, consistent data, and gives you time to concentrate on the detection problem.

About the simulator

The ENGF0001 simulator models the behaviour of a small bioreactor that has the same form as your bioreactor. It provides a realistic testbed for anomaly detection, because it combines process physics, sensor and actuator dynamics, and fault injection under realistic control conditions.

In detail (optional): The simulator runs quite a detailed simulation that respects the physics and chemistry of a real bioreactor and uses a set of coupled differential equations that represent heat transfer, fluid mixing, and acid and base dosing. The coupling means that a change in one quantity (for example, stirring speed) can affect others (for example, measured temperature). Sensor readings (temperature, pH and rotational speed) are generated from the underlying physical state, with appropriate ADC quantisation and noise. Actuators (heater, stirrer motor, acid and base pumps) are driven by a set of digital control signals.

Each control loop is governed by its own PID controller, tuned using an automatic relay-based method to produce stable and realistic closed-loop behaviour. This means that when a fault occurs, the control

system reacts to it. For example, if the temperature sensor reads too low, the controller increases the heater duty. Your detector will therefore see the same indirect and delayed effects that would appear in a real bioreactor.

Data streams

The ENGF0001 simulator publishes a summary message over MQTT once per second, in JSON format. Each message contains the mean, minimum and maximum temperature, pH and stirring speed (RPM) over the last second, the average actuator duty cycles, the volumes of acid and base dosed, the heater energy used, the number of pulses from the stirrer's optical speed sensor (`photoevents`, two per revolution), and the current setpoints. On labelled streams, each message also contains a `faults` field: `last_active` lists the faults that were active during the second, and `counts` gives, for each fault, how many of the samples in that second it was active for. A count smaller than the number of samples (given in `window.samples`) means the fault started or cleared part way through the second. You may use any field except `faults` as an input to your detector. You will subscribe to these data streams, train a detector on fault-free data, and evaluate its ability to identify faults. Your detector may run either:

- on an **ESP32** device connected directly to the broker, or
- in **Python** on your laptop, subscribing to the same MQTT topics.

During the final session you will demonstrate your detector working live.

Broker and access

Broker: `engf0001.cs.ucl.ac.uk` **Port:** `1883` (unencrypted)
Access: The broker is reachable only from the UCL network. If you are working off campus on your laptop, connect first using the UCL VPN[1]. The VPN does not help an ESP32, so if you use an ESP32 you will need to work on campus. Please give your MQTT client a unique client ID (for example, one that includes your team number), because two clients with the same ID will repeatedly disconnect each other. You only need to subscribe to the simulator topics. You have no need to publish to them, so please do not, as doing so could disrupt the streams for everyone.

[1] https://www.ucl.ac.uk/isd/services/get-connected/ucl-virtual-private-network-vpn

MQTT topics in use

Each topic corresponds to a different simulator configuration. Subscribe to topics of the form:

`bioreactor_sim/<DATA_STREAM>/telemetry/summary`

where `<DATA_STREAM>` is:

`<DATA_STREAM>`	**Fault labels that may appear**	**Notes**
`nofaults`	*none* (no faults active)	Clean baseline data for training or calibration.
`single_fault`	`therm_voltage_bias`	Temperature sensor bias only. The fault appears and clears at random. This is the stream the exercise requires.
`three_faults`	`therm_voltage_bias`, `ph_offset_bias`, `heater_power_loss`	Optional. Faults may overlap. Use this to test how general and robust your detector is.
`variable_setpoints`	`therm_voltage_bias`, `ph_offset_bias`, `heater_power_loss`	A stretch goal for those who want a real challenge, and the closest to reality. The same faults as `three_faults`, and the temperature, pH and RPM setpoints change on a schedule. There is no fault-free version of this stream, just as there would be no clean training data in a real plant, but you can use the fault labels to tell when no fault is present.

Operational notes Each fault type occurs and clears independently and at random. On average a fault type stays clear for 300 s (the mean time between failures, MTBF) and, once it has occurred, lasts for 60 s (the mean time to repair, MTTR). There is no minimum duration, so some faults last only a few seconds, and on streams with more than one fault type several faults may be present at once. On `single_fault` you should therefore expect to wait around five minutes for the first fault to appear, and the fault is present roughly one sixth of the time. The streams run continuously, 24 hours a day, to support asynchronous development and testing. In the `variable_setpoints` stream, setpoint changes are deterministic rather than random: the setpoints step to new values on the hour (UTC), following a pattern that repeats every 24 hours. This allows detectors to be tested for robustness against legitimate process changes as well as faults.

Fault label glossary

`therm_voltage_bias` Adds a voltage bias to the thermistor ADC input, so the measured temperature differs from the true temperature.

`ph_offset_bias` Adds an offset to the pH electrode signal, so the measured pH differs from the true pH by a steady amount.

`heater_power_loss` Reduces the heating power the heater delivers for a given duty cycle, as if the element were degrading. The temperature controller compensates by increasing the heater duty cycle.

Where faults show up The controllers try to keep each sensor reading at its setpoint. Once a controller has compensated for a fault, the sensor reading itself may look normal. It is worth thinking about which other signals the controller has had to change in order to achieve this.

Your task

1. Subscribe to one or more topics using an MQTT client or library (for example, `paho-mqtt` in Python, or the `PicoMQTT` library for the ESP32).
2. Train your detector on the `nofaults` stream to establish a baseline of normal behaviour.
3. Test it on `single_fault`. Optionally, test it on `three_faults` as well and, if you want a real challenge, on `variable_setpoints`.
4. Decide when to raise an anomaly flag based on deviations from the baseline. You must not use the fault labels in deciding what is anomalous, but you should use them to assess your decisions.
5. Keep score. Your detector should run on every published sample and should show the number of true positives (TP), true negatives (TN), false positives (FP) and false negatives (FN), counted second by second. Include a summary of this scoring in your final report. The precise requirements for the report will be on Moodle.
6. Look carefully at when your false positives and false negatives occur, relative to the start and end of each fault. Explain why they occur where they do, and consider whether scoring each second independently is a fair measure of a detector monitoring a physical process under feedback control. If you think it is not, propose a better scoring scheme, justify it, and report your results under both schemes.
7. Optionally, if you are working with `three_faults`, try to identify which fault is present as well as whether a fault is present.
8. Be prepared to demonstrate your detector live during the final session. For this, we may send faulty streams of data *without fault labels*, which is what you would expect in reality: it is the task of your detector to analyse the data correctly.

Expected outcomes

By completing this exercise you will:

- Understand how real-world process data can be monitored for faults using data-driven methods.
- Gain hands-on experience with MQTT messaging and real-time analysis.
- Implement and tune a simple but effective anomaly detector.
- Demonstrate a complete detection pipeline working on live bioreactor telemetry.